

CADF

Context-Agentic Development Framework

개발 방법론

v1.1 | 2026년 3월

기안: 박진웅

목차

1. 핵심 철학

핵심 명제

“사람은 컨텍스트를 큐레이션하고, AI 에이전트가 실행한다.”

CADF(Context-Agentic Development Framework)는 AI 에이전트를 핵심 실행자로 활용하는 프로젝트 프레임워크입니다.

기존 방법론에서 개발자는 요구사항을 읽고, 설계하고, 코딩하고, 테스트합니다. CADF에서는 이 흐름이 근본적으로 바뀝니다. 사람의 핵심 역할은 AI 에이전트가 소비할 수 있는 고품질 컨텍스트를 만들고 유지하는 것이고, 코드 생성 · 테스트 · 문서화 같은 실행은 AI 에이전트가 담당합니다.

여기서 “컨텍스트”란 단순한 문서가 아닙니다. 시스템 아키텍처, 도메인 규칙, 프로토콜 명세, 제약조건, 의사결정 이력 등을 AI가 즉시 참조할 수 있는 구조화된 지식 체계를 뜻합니다.

컨텍스트의 품질이 낮으면 AI 에이전트의 산출물도 낮아지고, 컨텍스트가 정밀하면 산출물의 품질이 극적으로 높아집니다. 이는 업계 조사와 실무 데이터로 동일하게 확인되는 사실입니다.

근거: 컨텍스트 품질과 AI 산출물 품질의 상관관계

Qodo 「State of AI Code Quality 2025」 보고서에 따르면, 컨텍스트를 수동 선택하는 개발자의 54%가 AI가 관련성을 놓친다고 응답했으나, 컨텍스트가 자동으로 선택되면 33%로, 세션 간 지속적으로 저장·재사용되면 16%로 급감합니다. 또한 Claude Code의 CLAUDE.md는 200줄 이하일 때 규칙 적용률 92% 이상, 400줄 초과 시 71%로 하락합니다 (SFEIR Institute). 즉, 컨텍스트의 구조와 품질이 AI 산출물의 신뢰도를 직접적으로 결정합니다.

1.1 CADF의 위치

CADF는 팀 구조를 바꾸지 않습니다. AI 연구자는 AI 연구를 하고, 플랫폼 개발자는 플랫폼을 만들고, HW 엔지니어는 로봇을 다룹니다. 다만 프로젝트가 시작될 때 모든 참여자가 따르는 공통 규약이 CADF입니다.

각 도메인 전문가가 자기 영역의 지식을 컨텍스트 파일로 내놓고, 그 컨텍스트를 기반으로 AI 에이전트가 통합 · 구현 · 테스트를 가속합니다. 사람 간 커뮤니케이션 비용을 컨텍스트가 대체합니다.

2. CADF의 3대 원칙

2.1 Context-First

코드를 쓰기 전에 컨텍스트를 먼저 쓴다. 새 기능을 시작할 때 가장 먼저 하는 일은 해당 기능의 도메인 지식, 제약조건, 인터페이스 규격을 컨텍스트 파일에 정리하는 것입니다. 이것이 곧 AI 에이전트에 대한 “발주서”가 됩니다.

2.2 Agent-as-Executor

AI 에이전트는 보조 도구가 아니라 핵심 실행자입니다. 코드 생성, 테스트 작성, 리팩토링, 문서 동기화를 AI가 수행하고, 사람은 산출물을 검증하고 컨텍스트를 개선합니다.

2.3 Feedback Crystallization

시뮬레이션 결과, 코드 리뷰, 운영 로그 등 모든 피드백은 반드시 컨텍스트에 결정(結晶)화됩니다. “이번에 이런 문제가 있었다”가 구두로 끝나지 않고, 컨텍스트 파일에 규칙으로 남아 AI 에이전트가 같은 실수를 반복하지 않게 합니다.

3. 프로젝트 생명주기: 3 Phase

모든 CADF 프로젝트는 규모와 관계없이 동일한 3단계를 거칩니다. 각 Phase의 기간은 프로젝트마다 유동적입니다.

Phase	명칭	핵심 활동
Phase 1	Assemble	컨텍스트 조립 — 도메인 전문가들이 각자의 지식을 컨텍스트 파일로 작성
Phase 2	Execute	Pulse 반복 실행 — AI 에이전트에게 컨텍스트 주입 → 생성 → 검증 → 결정화
Phase 3	Merge	결정화 & 환원 — 프로젝트 학습을 공통 컨텍스트에 반영

3.1 Phase 1 — Assemble (컨텍스트 조립)

프로젝트가 시작되면 가장 먼저 하는 일은 코딩이 아닙니다. 참여하는 도메인 전문가들이 각자의 지식을 컨텍스트 파일로 내놓는 것입니다.

Project Lead(PL)가 킥오프 미팅을 열고, 프로젝트의 목표와 범위를 정의합니다. 그 자리에서 각 전문가가 자기 도메인에서 “AI 에이전트가 알아야 할 것”을 약속합니다. 미팅이 끝나면 각자 컨텍스트 파일을 작성합니다.

예시 — “시뮬레이션 환경에서 로봇 팔 경로 계획 학습” 프로젝트:

- **AI 연구자:** 강화학습 알고리즘 선택 근거, 보상 함수 설계 제약, 학습 파이프라인 규격
- **플랫폼 개발자:** 시뮬레이션 엔진 API 명세, 데이터 포맷, 플랫폼 연동 인터페이스
- **HW 엔지니어:** 로봇 팔의 자유도, 토크 한계, 센서 스펙, 안전 제약조건

Phase 1 완료 기준

“이 컨텍스트만 읽으면 AI 에이전트가 프로젝트의 목표, 제약, 인터페이스를 이해할 수 있는가?” — 팀원 전원이 Yes라고 답하면 다음 Phase로.

3.2 Phase 2 — Execute (Pulse 반복 실행)

컨텍스트가 준비되면 Pulse라는 반복 사이클에 진입합니다. 하나의 Pulse는 3단계로 이루어집니다.

1. **Inject** — 컨텍스트를 AI 에이전트에 주입하고 작업을 지시합니다. 이 단계에서 WAT 패턴을 적용합니다: 먼저 Workflow(작업 흐름)를 명확히 정의하고, Agent 구성(어떤 AI 에이전트에게 어떤 역할을 맡길지)을 결정한 뒤, Tools(필요한 스크립트·MCP·Hook 등)를 조합합니다. 10분의 Workflow 정의가 수시간의 삽질을 줄입니다.

2. **Validate** — AI 에이전트의 산출물을 검증합니다. 시뮬레이션 실행, 코드 리뷰, 도메인 규칙 충족 여부를 확인합니다. 선택적으로 교차 검증도 활용합니다: Claude의 결과물을 ChatGPT나 Gemini 같은 다른 AI에게 보여주고 빠진 부분이나 잘못된 접근을 지적받으면 검증 품질이 높아집니다.
3. **Crystallize** — 이번 Pulse에서 발견한 것(실패한 접근법, 새 규칙, 도메인 간 충돌 해결)을 컨텍스트 파일에 기록합니다. Auto Memory에 있는 학습 중 팀 전체에 유효한 것은 공식 컨텍스트로 승격합니다.

핵심 원칙

“AI 결과물이 틀리면 코드가 아니라 컨텍스트를 고쳐라.” 컨텍스트가 불충분해서 AI가 잘못 생성한 것이므로, 원인을 컨텍스트에서 찾습니다.

Pulse 주기는 프로젝트마다 다릅니다. 빠른 PoC 프로젝트는 하루에 2–3 Pulse를 돌릴 수 있고, 복잡한 통합 프로젝트는 1 Pulse가 일주일일 수 있습니다. PL이 킥오프 때 초기 Pulse 주기를 정하고, 진행하면서 조정합니다.

3.3 Phase 3 — Merge (결정화 & 환원)

프로젝트 목표가 달성되면, 프로젝트에서 생성된 컨텍스트 중 플랫폼 전체에 가치 있는 지식을 공통 컨텍스트 베이스에 환원합니다.

예를 들어 “시뮬레이션에서 관절 토크가 실제의 87%만 반영된다”는 사실은 향후 다른 로봇 프로젝트에서도 유효한 지식입니다. 이런 것들이 공통 컨텍스트로 올라갑니다. 반면 해당 프로젝트에만 해당하는 임시 결정은 프로젝트 아카이브로 보관합니다.

Phase 3 완료 기준

“이 프로젝트를 6개월 뒤 다시 한다면, 컨텍스트만으로 80% 이상 재현 가능한가?”

4. 프로젝트 역할

CADF에서 도메인 전문가는 자기 직함을 유지합니다. 대신 프로젝트 안에서 두 가지 프로젝트 역할이 있습니다.

역할	설명
Project Lead (PL)	프로젝트를 제안한 사람이 맡습니다. Kickoff 미팅 주관, Pulse 주기 결정, CLAUDE.md 루트 파일 관리, 컨텍스트 정합성 최종 확인. 전담이 아니라 자기 도메인 작업을 하면서 걸임합니다.
Domain Contributor (DC)	나머지 참여자. 자기 도메인의 컨텍스트를 작성하고, 해당 영역에서 AI 에이전트를 활용해 실행하고, 결과를 검증합니다.

5명 이하 팀에서 한 사람이 여러 프로젝트의 DC가 될 수 있고, 동시에 자기가 제안한 프로젝트에서는 PL이 될 수 있습니다.

5. 컨텍스트 관리 체계

CADF의 컨텍스트 관리는 Claude Code의 메모리 메커니즘에 네이티브하게 설계되었습니다. Claude Code는 세션 시작 시 **CLAUDE.md** 파일과 **.claude/rules/** 디렉토리를 자동으로 읽습니다. 이 메커니즘을 최대한 활용하는 구조입니다.

5.1 디렉토리 구조

```

project-root/
├── CLAUDE.md                # PL만 수정, 200줄 이하
├── CLAUDE.local.md          # .gitignore, 개인 설정
├── .claude/
│   └── rules/
│       ├── simulation.md    # globs: ["src/simulation/**"]
│       ├── ml-pipeline.md   # globs: ["src/ml/**", "models/**"]
│       ├── robotics-hw.md   # globs: ["src/drivers/**"]
│       └── api-contracts.md # globs: ["src/api/**", "proto/**"]
├── docs/context/
│   ├── architecture.md      # @import 대상 상세 문서
│   ├── domain-robotics.md   # PL 관리
│   ├── domain-ml.md         # 담당자별 관리
│   ├── protocols.md
│   ├── experimental/
│   └── decisions/
└── docs/pulse-log/          # Pulse 기록
  
```

5.2 파일별 역할

CLAUDE.md (루트)

200줄 이하로 유지하며, “이 프로젝트가 무엇이고 어떤 규칙을 따르는가”의 요약만 담습니다. 상세 내용은 `@docs/context/architecture.md` 같은 `import`로 필요할 때만 불러옵니다. Claude Code는 200줄 이하일 때 규칙 적용률이 92% 이상이지만, 400줄을 넘으면 71%로 떨어집니다.

.claude/rules/ (도메인별 규칙)

glob 패턴을 활용해 특정 파일 경로에만 규칙을 적용합니다. 예를 들어 `simulation.md`는 `src/simulation/` 하위 파일을 작업할 때만 자동 적용됩니다. CADF의 “도메인 전문가가 자기 영역의 컨텍스트를 작성한다”는 원칙과 정확히 맞아떨어집니다.

docs/context/ (상세 지식 저장소)

Claude Code가 자동으로 읽지 않지만, `@import`나 명시적 참조를 통해 필요할 때 접근하는 깊은 지식 저장소입니다. 아키텍처 상세, 도메인 지식, 프로토콜 명세, ADR(의사결정 기록) 등이 여기에 위치합니다.

CLAUDE.local.md (개인 설정)

.gitignore에 포함되며, 각 팀원의 AI 에이전트 사용 습관(선호 응답 언어, 코드 스타일 미세 조정 등)을 담습니다. 팀 공유 규칙은 절대 여기에 넣지 않습니다.

6. 컨텍스트 파일 템플릿

6.1 도메인 컨텍스트 템플릿

도메인 전문가가 컨텍스트를 작성할 때 사용하는 공통 포맷입니다.

```
# [도메인명] Context for [프로젝트명]

## Purpose
AI 에이전트가 이 도메인에서 알아야 할 핵심 지식 (1~2문장)

## MUST (반드시 지켜야 할 것)
- ...

## MUST NOT (절대 하면 안 되는 것)
- ...

## PREFER (가능하면 따를 것)
- ...

## Current State
현재 확정된 사실과 설계

## Open Questions (AI가 임의 결정 금지)
- ...

## History
- [날짜] 변경 내용 - 이유
```

6.2 Pulse 기록 템플릿

```
# Pulse [번호] - [날짜 범위]

## 목표
이번 Pulse에서 달성하려 한 것

## AI 에이전트 활용 내역
- [도구명] + [작업] → [결과 요약]

## 발견 사항
- 컨텍스트 보강 필요: ...
- 새로운 제약조건: ...
- 실패한 접근법: ...

## 컨텍스트 변경
- [파일명] - [변경 내용]
```

6.3 .claude/rules/ 파일 템플릿

```
---
globs: ["src/simulation/**/*.py", "tests/simulation/**"]
---

# Simulation Domain Rules

## MUST
- ...

## MUST NOT
- ...

## PREFER
- ...
```

7. Git 협업 규칙

CADF의 컨텍스트 파일은 **Git**으로 버전 관리됩니다. 다음 규칙을 통해 팀원 간 충돌을 구조적으로 방지합니다.

7.1 컨텍스트 파일 담당자 1인 원칙

각 `.claude/rules/` 파일과 `docs/context/` 파일에는 담당 도메인 전문가가 지정되어 있고, 해당 담당자만 수정합니다. 다른 사람이 수정이 필요하면 담당자에게 요청합니다.

파일	담당	수정 권한
CLAUDE.md	Project Lead	PL 전용
<code>.claude/rules/simulation.md</code>	시뮬레이션 전문가	담당자 전용
<code>.claude/rules/ml-pipeline.md</code>	AI 연구자	담당자 전용
<code>.claude/rules/robotics-hw.md</code>	HW 엔지니어	담당자 전용
<code>.claude/rules/api-contracts.md</code>	플랫폼 개발자	담당자 전용

7.2 컨텍스트 변경 PR 필수

코드만 바뀔 PR은 일반 리뷰로 처리하지만, 컨텍스트 파일이 포함된 PR은 PL이 반드시 리뷰합니다. 컨텍스트의 일관성을 지키기 위함입니다.

7.3 Auto Memory → 공식 컨텍스트 승격

Claude Code의 Auto Memory는 개인 디렉토리(`~/.claude/`)에 저장되어 Git 추적 대상이 아닙니다. 팀원 A의 로컬에서 Claude Code가 학습한 패턴이 팀원 B에게는 전혀 공유되지 않습니다.

따라서 각 Pulse의 Crystallize 시점에 팀원이 자신의 Auto Memory를 리뷰하고, 팀 전체에 유효한 학습은 `.claude/rules/`나 `docs/context/`에 명시적으로 옮깁니다.

7.4 실험적 컨텍스트 격리

확정되지 않은 설계 변경은 `docs/context/experimental/`에 따로 두고, 확정 시 정식 위치로 이동합니다. 실험이 무산되면 컨텍스트 변경도 함께 롤백됩니다.

7.5 Git Worktree 활용

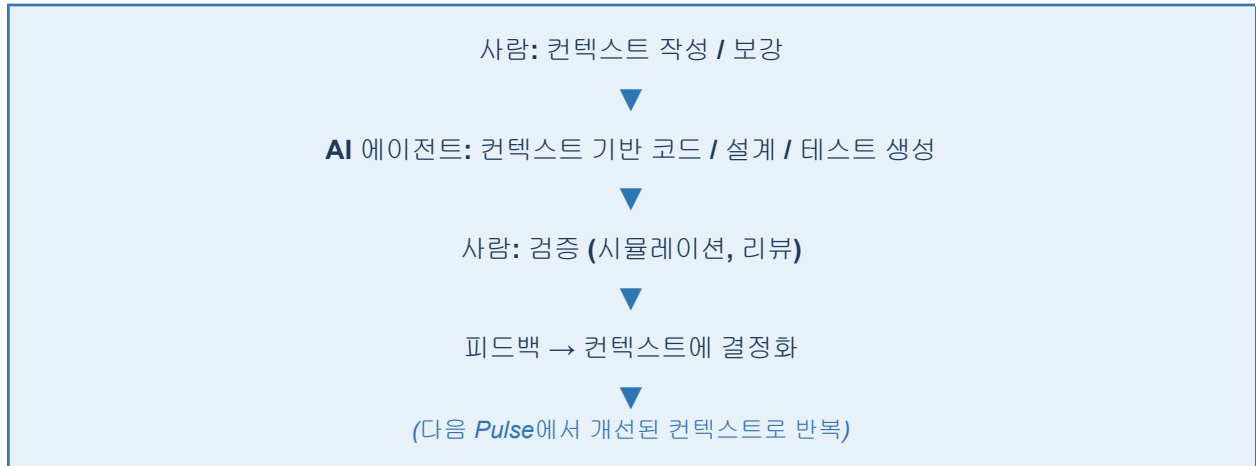
Claude Code는 Git **worktree**를 지원해 여러 브랜치에서 독립적인 세션을 동시에 실행할 수 있습니다. 두 팀원이 각자 **worktree**에서 작업하면서 서로 다른 방향으로 컨텍스트를 발전시킬 수 있으므로, 컨텍스트 파일 변경은 **main/develop** 브랜치에 마지 전 반드시 **PL**이 리뷰합니다.

8. 핵심 지표

지표	정의	의미
Context Coverage	프로젝트에서 AI 에이전트가 참조해야 할 도메인 중 컨텍스트가 작성된 비율	Assemble Phase의 완성도 측정
Agent First-Pass Rate	AI 에이전트가 생성한 산출물이 사람 수정 없이 검증을 통과하는 비율	컨텍스트 품질의 직접 지표
Crystallization Rate	각 Pulse에서 발견된 사항이 실제로 컨텍스트에 반영되는 비율	낮으면 같은 실수 반복

Agent First-Pass Rate가 낮다면 컨텍스트 품질이 부족하다는 신호입니다. 이 수치가 높아질수록 컨텍스트 품질이 좋다는 의미입니다.

9. 한눈에 보는 CADF



방법론의 핵심

CADF는 팀 구조를 바꾸지 않습니다. 기존 도메인 전문가들이 프로젝트에 참여할 때 따르는 공통 규약입니다. 컨텍스트의 품질이 곧 제품의 품질을 결정합니다.

10. Pulse 실행 기술: WAT 패턴

Pulse의 Inject 단계에서 “어떻게 AI 에이전트에게 작업을 지시할 것인가”에 대한 구체적 기술입니다. WAT는 Workflow-Agents-Tools의 약자로, 복잡한 작업을 분해하고 실행하는 3단계 패턴입니다.

10.1 W — Workflow (작업 흐름 정의)

코드를 쓰기 전에 작업 흐름을 평문(한국어/영어)으로 먼저 정의합니다. Plan Mode를 활용해 AI 에이전트에게 계획을 제시하게 하고, 리뷰한 뒤 실행으로 넘어갑니다. 10분의 Workflow 정의가 수시간의 삽질을 줄입니다.

```
# Workflow 예시: 로봇 팔 경로 계획 학습 파이프라인
1. MuJoCo 환경에서 URDF 로드 및 시뮬레이션 초기화
2. 보상 함수 설계 및 단위 테스트 작성
3. PPO 학습 루프 구현 및 시험 실행
4. 학습 결과 시각화 및 평가 리포트 생성
5. 각 단계마다 테스트 작성 후 통과 확인
```

10.2 A — Agents (에이전트 구성)

AI 에이전트의 역할을 정의합니다. Claude Code의 Sub-agent를 활용하면 역할을 분리하여 병렬 처리할 수 있습니다. 또한 에러가 발생하면 에이전트가 스스로 로그를 분석하고 수정하는 Self-Healing을 활용합니다.

- **Main Agent:** 핵심 기능 구현 (경로 계획 알고리즘)
- **Sub-agent A:** 테스트 작성 및 실행
- **Sub-agent B:** 문서 업데이트 및 컨텍스트 동기화
- **Sub-agent C:** 코드 린트 및 타입 체크

10.3 T — Tools (도구 조합)

작업을 잘 수행할 수 있도록 작고 원자적인 도구들을 조합합니다. 하나의 거대한 스크립트보다 작은 단위의 스크립트 여러 개가 훨씬 낫습니다 (Tool Atomicity).

- **Bash 스크립트:** 빌드, 테스트, DB 마이그레이션 등 반복 작업을 개별 스크립트로 분리
- **MCP 서버:** 외부 API(GitHub, 시뮬레이션 엔진 등) 연동. 단, 안 쓰는 MCP는 비활성화하여 토큰 절약
- **Hooks:** 커밋/저장 시 자동으로 린트·테스트 실행

핵심은 AI의 추론과 코드 실행을 분리하는 것입니다. AI 에이전트에게 생각하게 하고, 실행은 별도 도구에 맡기면 복잡한 작업도 안정적으로 관리할 수 있습니다.

11. 방법론의 진화 원칙

CADF는 “2025–2026년 스냅샷”으로 고정되면 안 됩니다. AI 도구가 빠르게 발전하고 있으므로, 방법론 자체도 진화해야 합니다.

11.1 불변 요소 vs. 가변 요소

불변 (철학적 원칙)	가변 (도구 종속적 요소)
구조화된 지식 관리가 결과 품질을 결정한다	CLAUDE.md 200줄 제한, glob 패턴, @import 전략
피드백을 체계적으로 축적해야 한다 (Crystallization)	특정 AI 도구의 메모리 메커니즘 (Auto Memory 등)
프로젝트 단위 협업 규약이 필요하다	세션 관리 (/clear, /compact 등) 전술
목표·제약·도메인 지식의 명확한 정의가 중요하다	WAT 패턴의 구체적 적용 방식

왼쪽 컬럼의 원칙은 AI가 발전해도 유효합니다. 오른쪽 컬럼의 요소는 도구가 바뀌면 함께 바뀔야 합니다.

11.2 개정 방식

CADF를 버전으로 관리합니다. 현재는 v1.0이며, 다음 기준으로 개정합니다.

- 정기 검토: 매 Horizon(월간) 리뷰 시 “이 방법론의 어떤 부분이 여전히 유효한가?”를 점검 항목에 포함
- 도구 변경 시: AI 도구의 주요 업데이트(컨텍스트 윈도우 확장, 새 협업 기능 등)가 있을 때 가변 요소를 업데이트
- 결정 기록: 방법론 변경 이유를 docs/context/decisions/에 ADR로 기록하여 변경 이력을 보존

CADF의 Crystallize를 CADF 자체에 적용

CADF가 말하는 피드백 결정화를 방법론 자체에도 적용합니다. “이 방법론의 어떤 부분이 실제로 도움이 되었는가?”를 주기적으로 점검하고, 발견사항을 방법론에 반영합니다.

부록 A. 개인 작업 가이드라인

CADF는 프로젝트 레벨의 프레임워크입니다. 아래는 각 팀원이 AI 에이전트를 실무에서 활용할 때 따를 개인 수준의 실행 팁입니다.

A.1 세션 관리

원칙	설명
한 세션 = 한 피처	하나의 세션에서 여러 기능을 연달아 구현하지 마세요. 신선한 컨텍스트가 부풀어진 컨텍스트보다 항상 낫습니다.
/clear 타이밍	한 단계의 작업이 완료되면 /clear 로 컨텍스트를 초기화하고 다음 단계를 새 세션으로 시작합니다.
/compact 활용	세션이 길어질 때는 /compact 로 대화 내용을 압축합니다. 단, 50% 컨텍스트 사용 전에 수동으로 실행하는 것이 좋습니다.
토큰 모니터링	/context 로 토큰 사용량을 주기적으로 확인하고, 안 쓰는 MCP는 /mcp 에서 비활성화합니다.

A.2 코딩 습관

원칙	설명
Plan Mode 먼저	큰 변경 작업은 반드시 Plan Mode (Shift+Tab)로 설계부터. Plan 없이 바로 실행하면 엉뚱한 방향으로 코드가 대량 수정될 수 있습니다.
TDD 루프	작은 변경 → 테스트 → 린트 → 커밋 → 반복. 문제가 생겨도 마지막 커밋으로 돌아가면 됩니다.
Thinking 로그 감시	AI가 생각하는 과정을 무시하지 마세요. 잘못된 가정을 발견하면 즉시 Escape 로 중단합니다. 초반에 잡는 것이 핵심입니다.
에러 로그 그대로	에러를 해석해서 설명하지 마세요. 에러 로그를 통째로 붙여넣으세요. 사람이 해석하면 오히려 정보가 빠질 수 있습니다.

A.3 작업 연속성 유지

별도의 **TODO.md**를 만들지 마세요. CADF에서는 이미 **Pulse** 기록(docs/pulse-log/)이 작업 추적 역할을 합니다. 할 일을 두 곳에 적으면 오히려 혼란이 생깁니다.

실전 방법: **Pulse** 기록의 “목표” 섹션에 세부 태스크를 체크리스트로 관리하세요. 세션 종료 시 “**Pulse** 기록 업데이트해줘”로 진행 상황을 반영하면, 다음 세션에서 해당 **Pulse** 기록을 읽는 것만으로 연속성이 유지됩니다.

A.4 컨텍스트 오염 방지

무거운 데이터 처리(CSV 파싱, 로그 분석 등)를 대화 안에서 시키면 컨텍스트가 오염됩니다. AI 에이전트에게 스크립트를 작성하게 하고, 결과 요약만 돌려받으세요.